

1 Dynamic Programming Solution to Number Partition

§1.1 Algorithm

1. As stated in the question, $b = \sum_{i=1}^n a_i$. Create a two-dimensional array DP that has dimensions $(n + 1) \times (b + 1)$. We call the array formed from the k th row of DP to be DP_k . The k in DP_k denotes that we are only considering the first k numbers in the input sequence of non-negative integers A . The algorithm will fill in the cells of DP iteratively, starting from the $k = 1$ row all the way to the $k = n$ row.
2. For the k th iteration, define $DP_k[j]$ to be a boolean variable. $DP_k[j]$ is True if there exists a subset of the first k elements that adds to j . If there does not exist such a subset, set $DP_k[j]$ to be False.
3. Our base case is when we do not consider any numbers from A ; this allows us to fill out the row array DP_0 . We do not need any numbers to create a partition with residue 0, so $DP_0[0] = \text{True}$. However, it is not possible to create a non-zero residue with no numbers from A , so $DP_0[i] = \text{False}, \forall i \in \{1, \dots, b\}$.
4. Our recurrence on the $(k + 1)$ -th iteration only relies on the boolean values from the k th iteration. On the $(k + 1)$ -th iteration, we are considering the element a_{k+1} for the first time. In order for a subset that sums to j to be achievable, we just need to check if a subset that sums to j or $j - a_{k+1}$ was achievable with the first k elements in A .

This gives us the recurrence:

$$DP_{k+1}[j] = \begin{cases} \text{False} & \text{if } j < 0 \text{ or } j > b \\ \text{True} & \text{if } DP_k[j] = \text{True} \text{ or } DP_k[j - a_{k+1}] = \text{True} \\ \text{False} & \text{otherwise} \end{cases}$$

5. We run n iterations, with the k th iteration allowing the element a_k to be considered for the first time in our number partition. On the k th iteration, we fill in every boolean value of DP_k from $DP_k[0]$ to $DP_k[b]$.
6. Once the table is filled, look at the final row, DP_n . Find the index x such that $DP_n[x] = \text{True}$ and the absolute difference $|2x - b|$ is minimized. This x represents the sum of the positive (+1)-partition that yields the smallest possible residue.
7. To find the exact partition this corresponds to, we initialize an empty sequence of signs S of length n . Then, we iterate backwards from the bottom DP row $k = n$ up to DP

row 1 with a running target sum x :

- If $x \geq a_k$ and $DP_{k-1}[x - a_k] = \text{True}$, it means the element a_k was included in this subset. Set $s_k = +1$, and update our target sum: $x = x - a_k$.
- Otherwise, it must have been that $DP_{k-1}[x] = \text{True}$. The element a_k was not in the $+1$ subset, meaning it belongs in the -1 subset. Set $s_k = -1$, and leave x unchanged.

We then return the sequence of signs $S = (s_1, \dots, s_n)$.

§1.2 Runtime

There are $(n + 1) \times (b + 1) = O(nb)$ subproblems. Each subproblem requires $O(1)$ time to solve because we only require a constant number of array accesses which we assume take $O(1)$ time. This means that to fill out the entire DP array, we require $O(nb)$ time. To obtain our solution, we need to find the value x such that $DP_n[x] = \text{True}$ and $|2x - b|$ is minimized. The n th row of DP (DP_n) contains $b + 1$ elements, and so checking all values and determining this value requires $O(b)$ time. Finally, our array-to-solution iteration checks 2 values in each row so it takes $O(n)$ time. Thus, the overall runtime of the algorithm is $O(nb)$, which is polynomial with respect to n and b . To reconstruct our solution, we also need to store a $(n + 1) \times (b + 1)$ dimensional array, which requires a space complexity of $O(nb)$, which is also polynomial with respect to n and b .

§1.3 Correctness

We will proceed with induction. We first see that the base case ($k = 0$) is correct because using no numbers, the only sum that can be achieved is 0. Therefore $DP_0[0] = \text{True}$ and all other values in that row are False.

Now suppose that the array is correctly constructed for the first k rows, we want to show that it will then be correctly constructed for the $(k + 1)$ th row. For each value in this row, $DP_{k+1}[j]$, we want it to store True if a subset of the first $k + 1$ elements of A can sum to j . The two cases for this to be true are if we include the $(k + 1)$ th element or don't. If we include it, then it must have been possible for a subset of the first k elements to sum to $j - a_{k+1}$ which means that $DP_k[j - a_{k+1}] = \text{True}$. If we do not include it, it must have been possible for the first k elements to sum to j so $DP_k[j] = \text{True}$. This is exactly our recurrence step so by induction, our array is constructed correctly.

We now see that finding the value x such that $DP_n[x] = \text{True}$ and $|2x - b|$ is minimized is the correct smallest residue. For the sake of contradiction, suppose we could find a residue smaller than $|2x - b|$. Let the sum of one of these better partitions be y . Then this means that we would have a residue of $|y - (b - y)| = |2y - b| < |2x - b|$. However this contradicts how we chose x so either $|2y - b| \geq |2x - b|$ in which case we do not have a better residue or $DP_n[y] = \text{False}$ and we cannot achieve this anyways. So, we see that x does indeed achieve the best residue possible.

Finally, we see that our backtracking will give us a valid sequence. In our backtracking algorithm, we make a sequence of signs such that one adds up to x and the other adds to $b - x$ therefore achieving our minimal residue $|x - (b - x)| = |2x - b|$ as explained above. To see that this correctly gives a sequence of signs such that it adds to x , we see that we set $s_k = +1$ if $x \geq a_k$ and $DP_{k-1}[x - a_k] = \text{True}$ which means that we add a value to the positive partition where we can still achieve the remaining sum from the remaining $k - 1$ elements. This will give all the necessary elements since we add an a_k as soon as possible. If doing this did not get us to a sum of x , then a sum of x would not be possible.

Therefore, we see that our algorithm correctly builds the DP matrix and turns this into a valid partition that attains the minimum residue so it is correct. ■

2 Karmarkar-Karp Algorithm Implementation in $O(n \log n)$ Time

§2.1 Implementation

1. Given an array A with n elements, insert every element in a binary max-heap. If we are constrained to only use binary min-heaps, we can simply negate the value of each element and insert every negated element into the min-heap. For the sake of simplicity, I will use a binary max-heap.
2. For $(n - 1)$ iterations, extract the two highest-value items in the binary max-heap and hold on to their two values. Then, take the absolute difference between these two values and insert this difference back into the binary max-heap. Continue this iterative process until there exists only one max-heap element remaining after the insertion of the absolute difference. Return this single remaining binary max-heap element as a possible residue value from the array A .

§2.2 Explanation

Adding all of the n elements of A into a binary max-heap makes the extraction of the highest-value elements and following insertion time-efficient. Within a single iteration, we delete the two-highest elements of that are remaining from A and then insert the absolute difference. This sequence of operations strictly removes one element from the heap; thus, we require at most $(n - 1)$ iterations to end up with a single remaining element that is our residue. This follows the specification of the Karmarkar-Karp algorithm.

§2.3 Runtime

The insertion of a single element into a binary max-heap takes $O(\log n)$, where n is the number of elements currently in the binary max-heap. This means that adding all n elements of A initially to the binary max-heap takes $O(n \log n)$ time. Then, within an iteration, recall we extract the two highest-value elements from the heap, which both take $O(\log n)$ time, and then insert the absolute difference into the binary max-heap. This takes a constant number (3 operations specifically) of binary max-heap operations, meaning one iteration of this sequence takes total $O(\log n)$ time. Since we perform $(n - 1) = O(n)$ iterations, the overall runtime of the $(n - 1)$ iterations takes $O(n \log n)$ time. Therefore, the overall runtime of this Karmarkar-Karp algorithm implementation is $O(n \log n)$ time. ■

3 Experimental Results

To validate our implementations, the various algorithms were run on 50 randomly-generated instances of the Number Partition problem. Below are the results across these 50 instances.

§3.1 Results

Table 1: Comparison of algorithm results across 50 runs

Idx	KK	RR	HC	SA	P-RR	P-HC	P-SA
0	250222	5861036	116058420	1036195224	188	126	168
1	128259	5248375	35409503	17938559	203	115	67
2	168356	206359774	84887430	1423416614	238	92	186
3	94074	1129511072	225471158	26640734	28	164	150
4	56560	164390016	945901758	165469266	452	374	136
5	18451	62668245	253159671	50770361	161	163	55
6	1214209	215641569	72458903	232363405	85	313	43
7	61725	184616315	18606047	485702117	3	37	57
8	92798	130395964	209321428	26351290	544	288	100
9	492130	106357890	74798264	179376418	344	88	172
10	3567916	114034736	40357738	180068850	136	98	350
11	2254441	615588309	97298883	163830231	253	133	35
12	280634	747140426	1064226	156166224	140	226	116
13	464513	251585471	230547623	11535917	157	93	97
14	13971	177156045	273700027	351888815	213	107	375
15	443673	36008701	889622603	165606953	353	51	123
16	17168	584185386	866473106	1558351256	270	212	198
17	28227	328046303	78318997	46004975	289	465	313
18	136312	646359922	19979646	101627382	10	212	58
19	53396	4344534	239600358	357755118	102	484	410
20	164848	7881966	42805592	432685688	54	208	24
21	95717	14602141	32801765	30671919	77	85	79
22	31318	382920614	65104142	562950660	102	16	158
23	131684	554486840	326985724	208188110	154	350	164
24	223096	16362150	284664208	808388830	82	292	474
25	172527	370708047	299208809	637737617	27	811	327
26	843536	175210982	221084542	46999100	100	120	798

Continued on next page

Run	KK	RR	HC	SA	P-RR	P-HC	P-SA
27	380420	72233266	311043196	58184558	40	122	30
28	1477451	367416539	113954507	177091971	373	69	179
29	22855	390252397	337373897	156232341	313	19	387
30	164701	19826257	90316443	3841215	55	131	23
31	230633	257835205	21170989	29429137	533	683	15
32	3196932	486803374	290374980	91190978	78	104	68
33	578013	697931047	240194835	194179511	203	163	37
34	670171	316298819	23572945	321926305	13	19	19
35	52170	114191460	51685500	697048084	146	900	262
36	49014	29277638	155845548	33300618	208	60	440
37	367044	97651620	541180734	204649460	106	60	334
38	1878509	142474863	363838457	100620897	19	7	133
39	312800	421973474	113244478	57741444	186	746	590
40	73468	303182248	301881052	480586142	510	102	4
41	3896	29674754	402333168	63216760	84	88	544
42	77714	563498112	277649038	255862568	192	222	50
43	333961	22753227	128836029	583041745	273	489	277
44	4972	18421920	373638600	123569150	190	696	162
45	529449	274367371	57501275	372238457	203	195	251
46	204047	79417417	121936167	374668779	49	361	309
47	36645	493985117	150415603	80268049	69	371	133
48	222865	208494867	374827637	288904513	93	97	75
49	679743	197645803	32170681	341361767	71	33	23

Statistic	KK	RR	HC	SA	P-RR	P-HC	P-SA
mean	460944.7	256865600	218413500	291076700	175.44	229.20	191.56
min	3896	4344534	1064226	3841215	3	7	4
25%	57851.25	65059500	66942830	59442610	77.25	89.00	57.25
50%	170441.5	191131100	153130600	178234200	150.00	132.00	143.00
75%	459303.0	379867500	297000400	368617600	231.75	307.75	301.00
max	3567916	1129511000	945901800	1558351000	544	900	798
time (s)	0.0002	0.4539	0.1202	0.1275	5.5001	5.0229	5.0297

§3.2 Discussion

We see that in every single run, the Karmarkar-Karp algorithm beats the non-prepartitioned versions of the repeated random, hill climbing, and simulated annealing algorithms since it has a smaller residue. We also notice that the Karmarkar-Karp algorithm takes significantly less time than the non-prepartitioned repeated random, hill climbing, and simulated annealing algorithms. This difference in timing is likely due to the fact that the repeated random, hill climbing, and simulated annealing algorithms ran for a large number of iterations that we specified (which we set to be at least 25000), whereas the Karmarkar algorithm runtime only depends on the number of elements in our initial number sequence (which was only set to 100).

Most importantly, we notice that prepartitioning yields drastically lower residues compared to the algorithms that do not employ prepartitioning. Specifically, we notice that the prepartitioned repeated random, hill climbing, and simulated annealing algorithms all significantly beat the Karmarkar-Karp algorithm and the non-prepartitioned versions of the three algorithms. This is because the prepartitioning includes running the Karmarkar-Karp algorithm, which allows us to compare residues with only refined solutions instead of random partitions.

Additionally, the Karmarkar-Karp algorithm is significantly faster than all other algorithms. For each of the three algorithms (repeated random, hill climbing, and simulated annealing), the prepartitioned version was about 4.9 seconds slower than its non-prepartitioned version. This is because in each iteration of the prepartitioned algorithms, we need to run the Karmarkar-Karp algorithm to obtain the residue solution, which takes $O(n \log n)$ time. This differs from the runtime within each iteration of the non-prepartitioned versions, where we can calculate the residue in $O(n)$ time. Doing this over at least 25000 iterations results in a large time discrepancy. Finally, the repeated random algorithm is slower than the hill climbing and simulated annealing in both the prepartitioned and not prepartitioned cases by about 0.4 seconds. This is likely due to the fact that in repeated random we need to initialize an entirely-new random sign assignment, while in the hill climbing and simulated annealing algorithms we only need to adjust the sign value of at most two indices for each iteration. Over the large number of iterations, this difference in the number of assignment operations can create a noticeable difference in runtime.

4 Karmarkar-Karp Algorithm as a Starting Point for Randomized Algorithms

§4.1 Explanation

To use the Karmarkar-Karp Algorithm as a starting point in the pre-partitioned algorithms, we would simply set $P = (1, 2, 3, \dots, n)$ since this would mean that $A' = A$. The next step in the algorithm is Karmarkar-Karp so we would be starting from the Karmarkar-Karp algorithm as intended. To use it as a starting point without pre-partitioning, we would pass the S implied by the algorithm as our starting S instead of a random solution. As described in the background, we could find this S by two-coloring the graph that arises from running the Karmarkar-Karp algorithm.

In the repeated random algorithm, we try a new random solution at every iteration. So, using the Karmarkar-Karp result means that our initial residue will likely be better than if we chose a random solution but it will not impact the runtime of the algorithm and will likely not impact the final result, so long as we run through enough iterations.

In the hill climbing algorithm, we check neighbors of our starting solution. The result of the Karmarkar-Karp algorithm is more likely to be a local minimum than a random solution is meaning that neighbors are less likely to result in an improvement. This means that the hill climbing algorithm will likely end up returning the Karmarkar-Karp solution as its final result, which we saw was quite an improvement from the results of the non-prepartitioned repeated random, hill climbing, and simulated annealing algorithms.

Similarly, the simulated annealing algorithm is unlikely to find neighbors that are better than the Karmarkar-Karp solution. Based on our cooling schedule, it is possible to begin looking at a different solution and potential find a different minimum. So, while it is more likely that we return the Karmarkar-Karp solution than on the standard simulated annealing algorithm, the cooling schedule makes it possible for us to escape the local minimum and find an even better solution.